

Problema del productor-consumidor
con caducidades y su resolución
con mutexes

Nicolas Corbal Capas
Hugo Veiga Couselo

Introducción

La concurrencia es una característica fundamental en los sistemas operativos modernos y un factor clave para mejorar el rendimiento. Sin embargo, el acceso concurrente a recursos compartidos hace necesario el uso de mecanismos de sincronización. En muchos casos se establece una jerarquía de prioridades de modo que aquellos elementos más prioritarios deben ser procesados preferentemente. En muchos casos, con tiempos de caducidad como los paquetes con TTL que tienen un tiempo de vida limitado y si permanecen en la cola de un router lo suficiente son descartados.

Dentro del clásico problema del productor-consumidor se puede entender como la situación en que varios productores produzcan items de distinta prioridad en una región de datos independiente de los otros productores, y un consumidor deba consumir preferentemente aquellos con prioridad más alta. Así el consumidor no solo debe acceder de forma segura a las regiones críticas, sino que también seleccionar la región crítica que desea acceder.

En el presente informe se documenta la implementación de una solución a este problema utilizando mutexes y variables de condición en lenguaje C, adaptando el código visto en clase para manejar varios buffers y simulando diferentes velocidades de ejecución mediante llamadas a la función `sleep()`. Se analiza el funcionamiento de la solución, las decisiones de diseño adoptadas y las conclusiones obtenidas tras su ejecución.

Forma de ejecución

Primeramente, para compilar el código un ejemplo de comando de compilación usando el compilador gcc es:

```
gcc -Wall -I ./ prod_cons_caducidad.c buffer.c -o ejecutable
```

Siendo el ejecutable ejecutado simplemente con:

```
./ejecutable
```

Funcionamiento del código

La responsabilidad del código es compartida por 3 archivos: `buffer.h`, `buffer.c` y `prod_cons_caducidad.c`. El archivo `buffer.c` contiene las funciones necesarias para la producción y la consumición, el archivo de cabecera el tipo de elemento que usaremos como forma de acceso común y `prod_cons_prioridad.c` la responsabilidad de la solución del problema.

En tipo de dato `datos` se recoge lo anterior necesario para la cola (un buffer y marcadores de inicio y fin) adicionalmente se añaden las variables de condición y el mutex como variante a usar variables globales. Además, se añade el entero `prioridad` para que cada productor sepa que prioridad tiene y en que buffer debe escribir. Otro tipo de dato es `item` que en vez de ser un entero es una estructura que contiene el entero a guardar, el tiempo de vida y el de creación para comprobar si está caducado.

El archivo `prod_cons_prioridad.c` se divide en funciones. La función `main` que se encarga de inicializar las variables y lanzar los hilos, así como de esperar a su finalización para la liberación de recursos y los hilos productores y consumidor. La producción del item esta vez implica la inicialización de la estructura con los tiempos de caducidad.

El funcionamiento del hilo productor es similar a un productor normal, espera un tiempo para simular un tiempo de producción, produce un item y adquiere el mutex. Si el mutex estuviese bloqueado el hilo se bloquea y espera a que el consumidor libere el mutex, luego realiza lo análogo con la variable de condición. Si hay espacio en el buffer inserta el item, sino la variable de condición se emplea para usar `pthread_cond_wait()` y llamar al consumidor. Luego avisa a la variable de condición con `pthread_cond_signal` y libera el mutex.

```

for(int i = 0; i < MAX_ITER; i++){
    sleep(1 + rand()%6);
    if(produce_item(arq, &suma, &it) < 1) break;

    pthread_mutex_lock(&args->mutex);
    while(args->cantidad >= N) pthread_cond_wait(&args->condp, &args->mutex);
    insert_item(it, args);
    printf("[%ld] P%d insire %d (caduc %ds)\n",
        tempo_ms(), args->prioridade, it.valor, it.caducidade);
    pthread_cond_signal(&args->condc);
    pthread_mutex_unlock(&args->mutex);
}

```

El consumidor es el mismo que en el problema de prioridad salvo que analiza el ítem leído y si $\text{fecha actual} - \text{fecha creación} > \text{caducidad}$ se descarta y no se contempla en la suma.

```

while(!(fin[0] && fin[1] && fin[2])){
    int feito = 0;
    for(int p = 0; p < NP && !feito; p++){
        if(fin[p]) continue;
        if(pthread_mutex_trylock(&buffers[p].mutex) != 0) continue;
        if(buffers[p].cantidad > 0){
            it = remove_item(&buffers[p]);
            pthread_cond_signal(&buffers[p].condp);
            pthread_mutex_unlock(&buffers[p].mutex);
            if(it.valor == 0){
                fin[p] = 1;
                printf("[%ld] Cons fin de P%d\n", tempo_ms(), p+1);
            } else if(tempo_ms() - it.t_creacion > (long)it.caducidade * 1000L){
                printf("\033[91m[%ld] Cons DESCARTADO %d de P%d (caduc %ds)\033[0m",
                    tempo_ms(), it.valor, p+1, it.caducidade);
                caducados[p]++;
            } else {
                printf("[%ld] Cons consume %d de P%d\n", tempo_ms(), it.valor, p+1);
                sumas[p] += it.valor;
                sleep(1 + rand()%3);
            }
            feito = 1;
        } else {
            pthread_mutex_unlock(&buffers[p].mutex);
        }
    }
    if(!feito) sched_yield();
}

```

Conclusiones

```
pc-hugo@hugo:~/Documentos/3_curso/2_cuatri/S0II/S0II_P3/opcional2$ ./prod_cons_caducidad
[2723452] P3 insire 13 (caduc 12s)
[2723462] Cons consume 13 de P3
[2724452] P2 insire 11 (caduc 1s)
[2725452] P3 insire 26 (caduc 1s)
[2726453] P3 insire 39 (caduc 6s)
[2726462] Cons DESCARTADO 11 de P2 (caduc 1s)
[2726462] Cons DESCARTADO 26 de P3 (caduc 1s)
```

La solución implementada asegura la ausencia de carreras críticas. Sin embargo, la penalización en el rendimiento que supone la serialización puede provocar conflictos cuando varios productores compiten por insertar elementos antes de que caduquen, especialmente si el consumidor no es capaz de atenderlos con la suficiente rapidez. La introducción de caducidades obliga a equilibrar la seguridad en el acceso concurrente con la eficiencia del procesamiento: un sistema demasiado estricto en la sincronización evita errores, pero puede incrementar el número de elementos descartados por expiración.

En ciertos ámbitos no supone un gran problema, pero en circunstancias como las mencionadas tales como los paquetes de red con especificador TTL puede suponer un desperdicio de la infraestructura y un problema serio de rendimiento. En otros planos como el industrial, no cumplir tiempos puede provocar paradas en la producción, pérdidas de producto y en resumen un gasto mayor. Incluso en muchos sistemas como robóticos que no pueden trabajar con datos de sensores muy desactualizados.

La implementación soluciona el problema base. La restricción temporal un problema de escalabilidad y de sistematización ya que no se puede promediar un comportamiento estable.